

Data 260 Homework – 1

Submitted by Sreeram Achutuni (ID: 019151808)

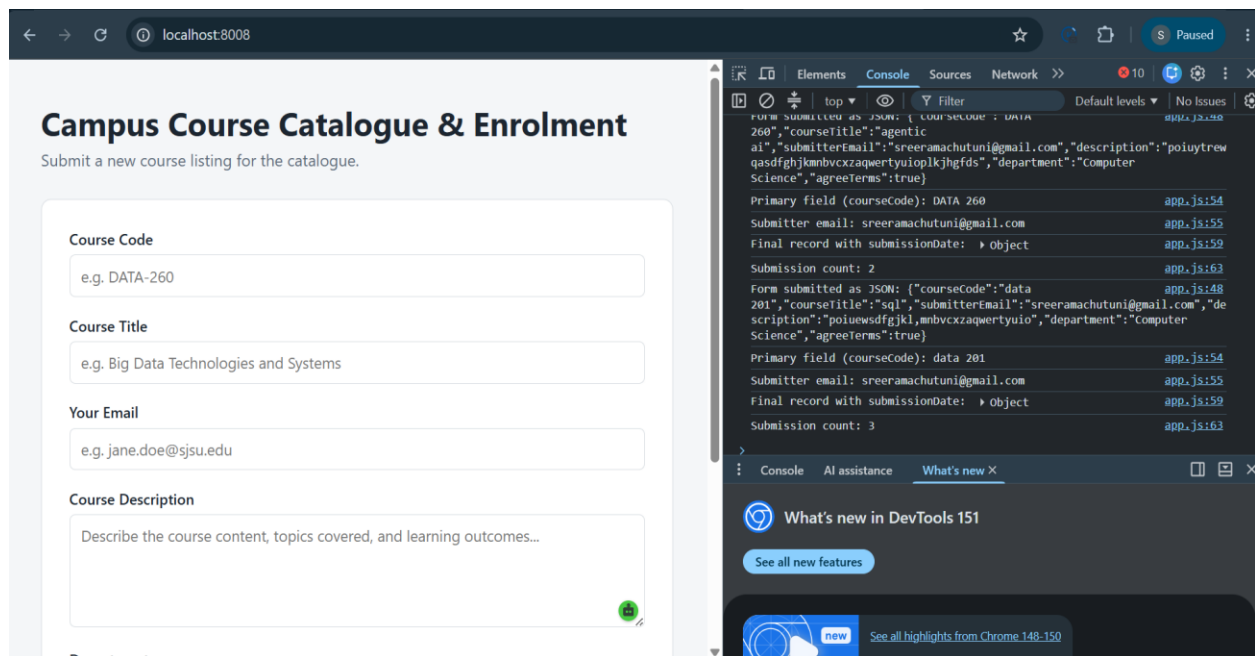
Section 0 — Personal Configuration

Value	Definition
SID4	1808
PORT_BASE	8008
PREFIX	s1808
SEED	1808
VERIFY_SEED	261808
DOMAIN_ID	0 → Campus course catalogue and enrolment
HARDWARE	Intel Core i7-11390H, 16GB RAM, integrated Intel Iris Xe Graphics (no dedicated GPU)
Local model	qwen2.5:1.5b-instruct (Ollama) — substituted for spec default qwen3:8b; see note below
Tagged commit hash	7b85b6d2bda149b8520e6e53c0b1751cb9b824d1
Git repo	https://github.com/sriram2930/Data260-1808

Part 1 (HTML, JavaScript & AWS)

Created an HTML page for Domain_ID = 0 (Campus course catalogue and enrolment). The form consists of:

- **Course Code:** the entity's primary field, required, autofocuses on page load
- **Course Title:** the entity's secondary field, required
- **Your Email:** required, type="email"
- **Course Description:** required textarea, validated at >25 characters
- **Department:** a dropdown with 4 domain-appropriate options (Computer Science, Data Science & AI, Business Analytics, Engineering)
- A checkbox labeled "I agree to the terms and conditions."
- **Submit Course Listing:** the submit button

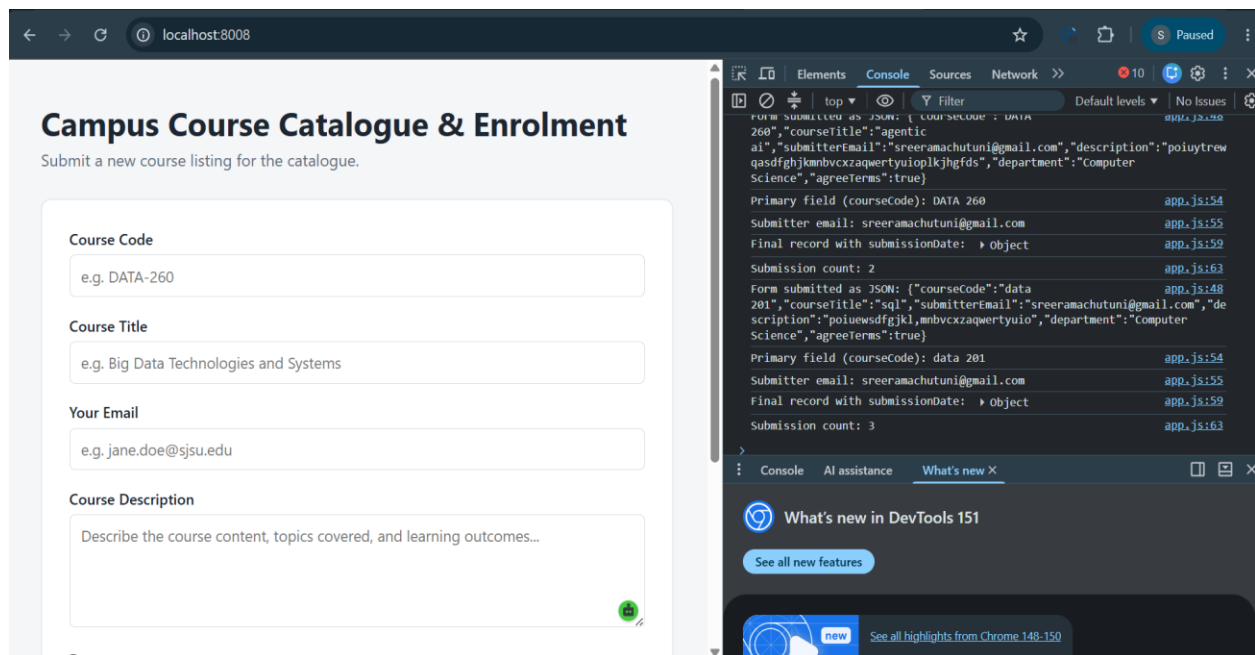


[illegible][illegible]

2 — JavaScript

When you click "Submit Course Listing", `event.preventDefault()` at the top of the submit handler prevents the default form-submission/page-reload behavior of the browser as this is a client-side-only demo with no backend to receive the data.

The script then grabs the current value of each of the six form fields straight off the DOM elements into a plain object and passes that object to `validateForm`, an arrow function that performs two checks in order: whether the description is longer than 25 characters, and whether the terms checkbox is checked. If either check fails, `alert()` tells you what's wrong and the function returns false, so nothing else runs (no logging, no reset).



Docker

Docker packages the app (`index.html`, `app.js`) into a portable container so it runs identically locally and on AWS. The Dockerfile starts from `nginx:alpine`, this is a minimal, production-realistic static-file server rather than a scripting-language dev server that copies in the two app files plus a custom `nginx.conf`, and listens on port **8008** (`PORT_BASE`) end-to-end: inside the container, in `docker run`'s port mapping, and later in the ECS task definition's container port and security-group rule, so the same port is used at every layer.

```
(.venv) PS C:\Users\User\Desktop\Data_260_HW-1\Part-2> docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
7e4efd28e2a8   data260-1808-hw1  "/docker-entrypoint..."  10 hours ago   Up 10 hours   0.0.0.0:8008->8008/tcp, [::]:8008->8008/tcp  data260-1808-hw1-container
(.venv) PS C:\Users\User\Desktop\Data_260_HW-1\Part-2>
```

AWS ECS Deployment

Rather than clicking through the console for each step, this deployment was carried out through the AWS CLI once it was installed and configured, using reproducible commands rather than a sequence of manual console actions.

The first issue arose while pushing the image to ECR. The image built successfully with docker build, but docker login against ECR from the local machine consistently failed with a 400 Bad Request, despite valid credentials. This was confirmed not to be a permissions issue by testing the same token against the ECR registry directly over HTTPS, which succeeded with a 200 OK. The failure was therefore isolated to a bug in Docker Desktop's login handling rather than an authentication problem. To work around it, the image was pushed to ECR from AWS CloudShell instead, which uses its own Docker installation and was unaffected.

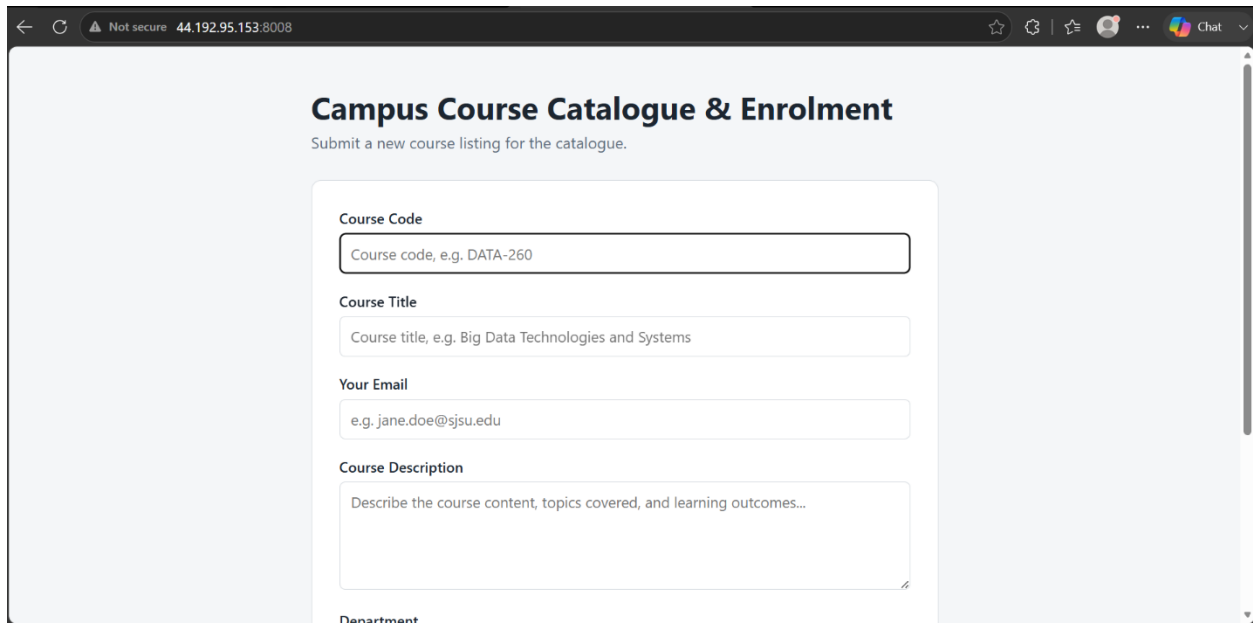
For networking, a security group (s1808-hw1-sg) was created with an inbound rule allowing TCP traffic on port 8008, matching the port the application listens on.

Creating the ECS cluster (s1808-hw1-cluster, AWS Fargate) initially failed with the error "Unable to assume the service linked role." This is a known first time account issue: the AWSServiceRoleForECS role had not yet been created for the account. Creating the role manually and retrying the cluster creation resolved the issue.

Configuring the task definition (s1808-hw1-task; Fargate, Linux/X86_64, 0.25 vCPU / 0.5 GB, container listening on port 8008, logging to CloudWatch) surfaced two further first time account gaps. The ecsTaskExecutionRole also did not exist and was created manually with the standard AmazonECSTaskExecutionRolePolicy attached. Tasks subsequently failed to start because they were unable to create their own CloudWatch log group; the managed policy grants permission to write logs and create log streams, but not to create the log group itself. Creating the log group manually resolved this, after which tasks started as expected.

The final component was the ECS service (s1808-hw1-service), configured with a desired count of 1, the default VPC, public subnets, the security group created earlier, and public IP assignment enabled to allow external access.

Once the task reached the RUNNING state, its public IP was verified with curl before being used for the report screenshot, returning HTTP 200 with the correct page title and confirming the deployment was serving the application correctly.



The screenshot shows a web browser window with a dark header bar. The address bar displays a 'Not secure' warning and the IP address '44.192.95.153:8008'. The browser's toolbar includes back, forward, and refresh buttons, along with icons for bookmarks, extensions, and a chat feature. The main content area has a light gray background. At the top, the title 'Campus Course Catalogue & Enrolment' is displayed in a bold, dark font, followed by the subtitle 'Submit a new course listing for the catalogue.' Below this, a white form box contains several input fields: 'Course Code' with a placeholder 'Course code, e.g. DATA-260', 'Course Title' with a placeholder 'Course title, e.g. Big Data Technologies and Systems', 'Your Email' with a placeholder 'e.g. jane.doe@jsu.edu', and 'Course Description' with a placeholder 'Describe the course content, topics covered, and learning outcomes...'. A 'Department' label is partially visible at the bottom of the form.

Campus Course Catalogue & Enrolment
Submit a new course listing for the catalogue.

Course Code
Course code, e.g. DATA-260

Course Title
Course title, e.g. Big Data Technologies and Systems

Your Email
e.g. jane.doe@jsu.edu

Course Description
Describe the course content, topics covered, and learning outcomes...

Department

Part 2 (Agentic AI)

Two tiny agents (Planner and Reviewer) that communicate through a local LLM served by Ollama, followed by a deterministic Finalizer step, producing exactly 3 topical tags and a summary of at most 25 words from an arbitrary title and content pair, printed as strict JSON.

Environment setup

A Python 3.12 virtual environment was created and langchain-ollama, pydantic, and their dependencies were installed into it. Python 3.12 was used specifically rather than the 3.14 already on this machine, since the assignment notes that Python 3.13 and later have compatibility issues with langchain's numpy dependency.

The model itself was pulled separately through the Ollama CLI, which runs as its own background service and serves models over a local HTTP API that langchain-ollama then talks to. The spec's default model is qwen3:8b, but this machine has no dedicated GPU, only integrated Iris Xe graphics, so an 8B model was impractical for the volume of sequential calls Part 3 would require. qwen2.5:1.5b-instruct was pulled instead, from the same model family, as a documented substitution.

Architecture of the agents

PlannerOutput and ReviewerOutput are Pydantic models that define the exact JSON shape each agent must return. These are passed to ChatOllama.with_structured_output, which uses Ollama's JSON schema constrained generation so the model is forced to emit valid JSON in the expected shape rather than free text. Two system prompt strings, one for the Planner and one for the Reviewer, are written generically around "the given title and content" with no reference to any particular domain, so the same code applies regardless of what entity type is being tagged.

run_pipeline() orchestrates the sequence: it calls the Planner with only the raw title and content, then calls the Reviewer with the same title and content plus the Planner's draft tags and summary appended to the prompt, and finally runs a Finalizer step. The Finalizer is not a third model call. It is a short, deterministic Python function that enforces the output contract regardless of what the model produced: it de-duplicates and hard-caps the tag list to exactly 3 entries, and hard-truncates the summary to 25 words if it runs over. This

guarantees the published JSON always meets the assignment's constraints, even when a model call does not.

How each agent works, in my own words

The Planner receives only the title and content and is asked to produce exactly 3 topical tags and a one sentence summary of at most 25 words, based strictly on the given text. The Reviewer receives the same title and content plus the Planner's draft, and is asked to critically check it: are the tags specific rather than generic, and does the summary respect the length limit and stay faithful to the content. It is instructed to return the draft unchanged if it already meets these standards, or a corrected version if not.

Comparing the Reviewer's output against the Planner's output is what answers whether the Reviewer changed anything. In practice, the Reviewer's own self assessment is not fully reliable. On one run, it marked a draft as approved with no changes needed while its own summary was still well over the 25 word limit it had just been asked to check. It had reworded the summary slightly but had not actually verified the constraint it claimed to have checked. The Finalizer caught this and truncated the summary regardless, which is exactly why that step exists as deterministic code rather than a third model call: it does not depend on the model correctly self-certifying its own output.

Result

Console output

```
(.venv) PS C:\Users\User\Desktop\Data_260_M-1> cd Part-2
(.venv) PS C:\Users\User\Desktop\Data_260_M-1\Part-2> ollama list
NAME          ID          SIZE      MODIFIED
qwen2.5:1.5b-instruct  65ec86548149  986 MB    2 hours ago
(.venv) PS C:\Users\User\Desktop\Data_260_M-1\Part-2> .venv\scripts\python agents_demo.py --input-file sample_input.json --temperature 0.7
=== INPUT ===
{
  "title": "DATA-260: Data Systems for Machine Learning",
  "content": "This course covers the design and operation of data pipelines that feed machine-learning systems: schema design, batch and streaming ingestion, feature stores, data versioning, and monitoring for data drift. Students build a small pipeline that ingests raw records, validates and transforms them, and serves features to a downstream model, while comparing batch and streaming trade-offs and discussing failure modes such as schema evolution and late-arriving data."
}

=== PLANNER OUTPUT ===
{
  "tags": [
    "machine learning",
    "data pipelines",
    "pipelines design"
  ],
  "summary": "This course teaches how to design and operate data pipelines for machine learning systems, including schema design, ingestion, feature stores, versioning, and monitoring."
}

=== REVIEWER OUTPUT ===
{
  "tags": [
    "machine learning",
    "data pipelines",
    "pipelines design"
  ],
  "summary": "This course focuses on designing and operating data pipelines for machine learning systems, including schema design, ingestion, feature stores, versioning, and monitoring. Students build a small pipeline that ingests raw records, validates and transforms them, and serves features to a downstream model.",
  "approved": true,
  "changes": "no changes needed."
}

=== FINALIZED / PUBLISH OUTPUT ===
{
  "tags": [
    "machine learning",
    "data pipelines",
    "pipelines design"
  ],
  "summary": "This course focuses on designing and operating data pipelines for machine learning systems, including schema design, ingestion, feature stores, versioning, and monitoring. Students build a",
  "model": "qwen2.5:1.5b-instruct",
  "temperature": 0.7,
  "reviewer_approved": true,
  "reviewer_changes": "no changes needed.",
  "finalizer_notes": [
    "truncated summary from 42 to 25 words"
  ],
  "generated_at": "2026-08-30T22:24:02.065873+00:00"
}

(pipeline latency: 12637.3 ms)
```

- **Q1 (3 final tags):** ["machine learning", "data pipelines", "pipelines design"]
- **Q2 (final summary, ≤25 words):** *"This course focuses on designing and operating data pipelines for machine learning systems, including schema design, ingestion, feature stores, versioning, and monitoring. Students build a"* this is exactly 25 words.
- **Q3 (did the Reviewer change anything?): No, not meaningfully.** It returned the same 3 tags and only lightly reworded the summary, marking is approved but the summary was still 42 words, over the limit it was told to check. So the Reviewer didn't actually fix the real problem, but the deterministic Finalizer did

Part 3 (Measuring Non-Determinism)

The same fixed input was run through the full agents_demo.py pipeline 40 times, 20 at temperature 0.7 and 20 at temperature 0.0, to measure with real numbers rather than assumption how much a local LLM's output actually varies from run to run at each setting.

Fixed input

Saved once, unchanged, as reports/hw01/cases/nondeterminism_input.json:

```
{  
  "title": "DATA-260: Data Systems for Machine Learning",  
  "content": "This course covers the design and operation of data pipelines that feed machine-learning systems: schema design, batch and streaming ingestion, feature stores, data versioning, and monitoring for data drift. Students build a small pipeline that ingests raw records, validates and transforms them, and serves features to a downstream model, while comparing batch and streaming trade-offs and discussing failure modes such as schema evolution and late-arriving data."  
}
```

Driver script

run_nondeterminism.py reuses run_pipeline() from agents_demo.py directly, looping it 20 times per temperature and timing each run. For each temperature it computes the number of distinct tag sets produced, the tags that appeared in every run, the tags that appeared in exactly one run, and latency percentiles across the 20 runs. A tag set is compared independent of order and case, since the experiment is measuring topical consistency rather than the exact printed order of the tags. All 40 raw records, including tags, summary, latency, and Finalizer notes, are written to reports/hw01/raw/nondeterminism_runs.json and .csv, with the aggregated statistics written to reports/hw01/raw/nondeterminism_summary.json.

```

(pipeline latency: 17637.3 ms) .venv\Scripts\python.exe run_nondeterminism.py | Tee-Object -FilePath reports\hw01\RUN_LOG_part3.txt
>> env) PS C:\Users\User\Desktop\Data_260 HW-1\Part-2>
=== DATA-260 HW1 Part 3 ù non-determinism run started 2026-08-30 17:10:18 ===
Fixed input: C:\Users\User\Desktop\Data_260 HW-1\Part-2\reports\hw01\cases\nondeterminism_input.json
Model: qwen2.5:1.5b-instruct | Runs per temperature: 20 | Temperatures: [0.7, 0.0]

=== Running 20 runs at temperature=0.7 ===
[2026-08-30T17:10:18] [temp=0.7] run 1/20 -> tags=['data pipelines', 'machine learning', 'pipeline design'] latency=16409.7ms
[2026-08-30T17:10:35] [temp=0.7] run 2/20 -> tags=['machine learning', 'data systems', 'pipeline design'] latency=7539.6ms
[2026-08-30T17:10:43] [temp=0.7] run 3/20 -> tags=['Machine Learning', 'Data Systems', 'Pipeline Design'] latency=7532.1ms
[2026-08-30T17:10:51] [temp=0.7] run 4/20 -> tags=['Machine Learning', 'Data Pipelines', 'Data Systems'] latency=7945.1ms
[2026-08-30T17:10:59] [temp=0.7] run 5/20 -> tags=['data systems', 'machine learning pipelines', 'data ingestion'] latency=7887.0ms
[2026-08-30T17:11:08] [temp=0.7] run 6/20 -> tags=['data pipelines', 'machine learning', 'schema design'] latency=7325.2ms
[2026-08-30T17:11:16] [temp=0.7] run 7/20 -> tags=['machine learning', 'data systems', 'pipeline design'] latency=8102.3ms
[2026-08-30T17:11:24] [temp=0.7] run 8/20 -> tags=['data pipelines', 'machine learning', 'data ingestion'] latency=8076.6ms
[2026-08-30T17:11:33] [temp=0.7] run 9/20 -> tags=['data pipelines', 'machine learning', 'data management'] latency=7478.0ms
[2026-08-30T17:11:41] [temp=0.7] run 10/20 -> tags=['Data Systems', 'Machine Learning', 'Pipeline Design'] latency=10555.5ms
[2026-08-30T17:11:52] [temp=0.7] run 11/20 -> tags=['machine learning', 'data pipelines', 'schema design'] latency=7639.2ms
[2026-08-30T17:12:00] [temp=0.7] run 12/20 -> tags=['Data Pipelines', 'Machine Learning', 'Ingestion'] latency=10491.4ms
[2026-08-30T17:12:11] [temp=0.7] run 13/20 -> tags=['data systems', 'machine learning', 'pipeline design'] latency=6354.5ms
[2026-08-30T17:12:18] [temp=0.7] run 14/20 -> tags=['data systems', 'machine learning', 'pipeline design'] latency=7534.3ms
[2026-08-30T17:12:26] [temp=0.7] run 15/20 -> tags=['machine learning', 'data pipelines', 'pipeline design'] latency=8246.1ms
[2026-08-30T17:12:34] [temp=0.7] run 16/20 -> tags=['data systems', 'machine learning', 'pipelines'] latency=12253.8ms
[2026-08-30T17:12:47] [temp=0.7] run 17/20 -> tags=['machine learning', 'data pipelines', 'pipeline design'] latency=9650.4ms
[2026-08-30T17:12:57] [temp=0.7] run 18/20 -> tags=['machine learning', 'data systems', 'pipelines'] latency=10878.7ms
[2026-08-30T17:13:09] [temp=0.7] run 19/20 -> tags=['data pipelines', 'machine learning', 'pipelining'] latency=11106.2ms
[2026-08-30T17:13:20] [temp=0.7] run 20/20 -> tags=['data systems', 'machine learning', 'pipeline design'] latency=7563.4ms

=== Running 20 runs at temperature=0.0 ===
[2026-08-30T17:13:28] [temp=0.0] run 1/20 -> tags=['machine learning', 'data pipelines', 'data systems'] latency=7671.5ms
[2026-08-30T17:13:36] [temp=0.0] run 2/20 -> tags=['machine learning', 'data pipelines', 'data systems'] latency=6975.1ms
[2026-08-30T17:13:44] [temp=0.0] run 3/20 -> tags=['machine learning', 'data pipelines', 'data systems'] latency=6917.9ms
[2026-08-30T17:13:51] [temp=0.0] run 4/20 -> tags=['machine learning', 'data pipelines', 'data systems'] latency=7106.2ms
[2026-08-30T17:13:59] [temp=0.0] run 5/20 -> tags=['machine learning', 'data pipelines', 'data systems'] latency=6960.1ms
[2026-08-30T17:14:06] [temp=0.0] run 6/20 -> tags=['machine learning', 'data pipelines', 'data systems'] latency=7250.6ms
[2026-08-30T17:14:14] [temp=0.0] run 7/20 -> tags=['machine learning', 'data pipelines', 'data systems'] latency=7511.0ms
[2026-08-30T17:14:22] [temp=0.0] run 8/20 -> tags=['machine learning', 'data pipelines', 'data systems'] latency=7089.5ms
[2026-08-30T17:14:30] [temp=0.0] run 9/20 -> tags=['machine learning', 'data pipelines', 'data systems'] latency=7180.0ms
[2026-08-30T17:14:37] [temp=0.0] run 10/20 -> tags=['machine learning', 'data pipelines', 'data systems'] latency=7073.0ms
[2026-08-30T17:14:45] [temp=0.0] run 11/20 -> tags=['machine learning', 'data pipelines', 'data systems'] latency=7043.0ms
[2026-08-30T17:14:52] [temp=0.0] run 12/20 -> tags=['machine learning', 'data pipelines', 'data systems'] latency=7116.5ms
[2026-08-30T17:15:00] [temp=0.0] run 13/20 -> tags=['machine learning', 'data pipelines', 'data systems'] latency=7140.4ms
[2026-08-30T17:15:08] [temp=0.0] run 14/20 -> tags=['machine learning', 'data pipelines', 'data systems'] latency=7138.5ms
[2026-08-30T17:15:15] [temp=0.0] run 15/20 -> tags=['machine learning', 'data pipelines', 'data systems'] latency=7403.0ms
[2026-08-30T17:15:23] [temp=0.0] run 16/20 -> tags=['machine learning', 'data pipelines', 'data systems'] latency=7224.5ms
[2026-08-30T17:15:31] [temp=0.0] run 17/20 -> tags=['machine learning', 'data pipelines', 'data systems'] latency=7381.6ms
[2026-08-30T17:15:39] [temp=0.0] run 18/20 -> tags=['machine learning', 'data pipelines', 'data systems'] latency=7103.4ms
[2026-08-30T17:15:47] [temp=0.0] run 19/20 -> tags=['machine learning', 'data pipelines', 'data systems'] latency=7021.3ms
[2026-08-30T17:15:55] [temp=0.0] run 20/20 -> tags=['machine learning', 'data pipelines', 'data systems'] latency=7047.5ms

=== SUMMARY ===

```

Results:

Metric	Temp = 0.7	Temp = 0
Distinct tag sets	10	1
Tags in all 20 runs	none	data pipelines, data systems, machine learning
Tags in exactly 1 run	data management, ingestion, machine learning pipelines, pipelining	none

Latency (ms)	Temp = 0.7	Temp = 0
p50	8076.6	7116.5
p95	12253.8	7551.0
P99	16409.7	7691.5

What two users sending identical input might see

At temperature 0.0, two users would see the exact same output every time. All 20 runs converged on one identical tag set, and latency was also the tightest and most predictable of the two settings, with p50 to p99 spanning only about 550 milliseconds. Temperature 0.0 effectively forces greedy decoding, so identical input reliably produces identical output.

At temperature 0.7, two users would very likely see genuinely different results. Out of 20 runs on the same input, 10 distinct tag sets came out, meaning half the runs produced a tag combination no other run produced, and no single tag appeared in all 20 runs. One user might see "data management" and "ingestion" as tags, while another user asking the identical question sees "pipelining" instead. The closest thing to a stable pattern is that most tags orbit the same handful of underlying concepts, machine learning, data pipelines, and data systems, expressed with different wording each time, rather than identical output.

Where this variation is acceptable, and where it is not

An example where this variation is acceptable is a suggested tags feature shown to a course catalogue submitter while they draft a listing, where the output is reviewed and edited by a person before anything is published. Here the variety at temperature 0.7 is actually useful. Different phrasings across attempts give the submitter more options to choose from, and because a human reviews the result before it goes live, run to run inconsistency never reaches an end user.

An example where this variation is not acceptable is using these tags to automatically route a course into search index categories with no human review. A student searching the catalogue for one of the exactly one run tags would find this course in some runs but not others, even though the underlying course description never changed, which is a confusing experience caused entirely by sampling randomness rather than any real change in the content. This is exactly the kind of case where temperature 0.0 should be used instead, since anything feeding an automated decision without a human checkpoint needs to be reproducible.

```
=== SUMMARY ===

--- temperature=0.7 ---
{
  "distinct_tag_sets": 10,
  "tags_in_all_runs": [],
  "tags_in_exactly_one_run": [
    "data management",
    "ingestion",
    "machine learning pipelines",
    "pipelining"
  ],
  "latency_p50_ms": 8076.6,
  "latency_p95_ms": 12253.8,
  "latency_p99_ms": 16409.7
}

--- temperature=0.0 ---
{
  "distinct_tag_sets": 1,
  "tags_in_all_runs": [
    "data pipelines",
    "data systems",
    "machine learning"
  ],
  "tags_in_exactly_one_run": [],
  "latency_p50_ms": 7116.5,
  "latency_p95_ms": 7511.0,
  "latency_p99_ms": 7671.5
}

Raw rows (CSV): C:\Users\User\Desktop\Data_260_HW-1\Part-2\reports\hw01\raw\nondeterminism_runs.csv
Raw rows (JSON): C:\Users\User\Desktop\Data_260_HW-1\Part-2\reports\hw01\raw\nondeterminism_runs.json
Summary: C:\Users\User\Desktop\Data_260_HW-1\Part-2\reports\hw01\raw\nondeterminism_summary.json

=== finished 2026-08-30 17:16:02 ===
(.venv) PS C:\Users\User\Desktop\Data_260_HW-1\Part-2> 
```

Part 4 (Model Client and Token Accounting)

A reusable model adapter module, a small CLI chat demo built on top of it, a system prompt file enforcing a strict response format, and real per-turn and cumulative token accounting.

src/model_client.py, the adapter

Defines `ModelClient`, exposing exactly one stable method, `complete(messages, tools=None)`, that every other script calling the model goes through. This adapter uses the raw `ollama` Python client rather than the `langchain_ollama` wrapper used in `agents_demo.py`, because Ollama's raw chat response carries the exact `prompt_eval_count` and `eval_count` fields needed for accurate per-turn token accounting. `complete()` normalizes the response into a dictionary containing the content, role, and a usage object with input, output, and total tokens for that turn, plus the raw provider response for anything not otherwise normalized. Nothing outside this file needs to know it is talking to Ollama specifically, so swapping the underlying provider later would only mean changing this one module.

AGENT.md, the strict review instruction

A system prompt file stating that whenever the assistant is asked to review code, it must respond in bullet points only, with no introductory sentence, no concluding paragraph, and no prose outside the bullets, and that a single "No issues found." bullet should be used only when there genuinely are no issues. It also states that this bullet only rule applies specifically to code review requests, and that other questions should be answered normally in plain prose.

hw1_client.py, the CLI demo

Loads `AGENT.md` as the system prompt, keeps its own conversation history as a plain list of role and content dictionaries, and calls `client.complete(history)` on every turn. After every model response it prints that turn's input, output, and total tokens. Typing `/stats` prints the turn count, cumulative input and output tokens, and the serialized length of the conversation history in characters, without appending anything to the history itself, since it is a read only command rather than a real conversational turn. On exit, whether by typing `/exit`, `/quit`, or pressing `Ctrl-D`, it prints the final cumulative input tokens, output tokens, and turn count.

Result

```
(.venv) PS C:\Users\User\Desktop\Data_260_HW-1\Part-2> Get-Content smoke_test_conversation.txt | .venv\Scripts\python.exe hwd_client.py | Tee-Object -FilePath reports\hw01\RUN_LOG_part4.txt
hwd_client.py - model=gpt2.5.1.5b-instruct - AGENT.md loaded as system prompt
Type a message, or /stats, or /exit (ctrl-D also exits).

> [assistant]: - The function 'divide' is named correctly.
- The function has a docstring that explains its purpose.
- The function takes two arguments, 'a' and 'b', which suggests it could handle division by zero. It does not include a check for 'b != 0', which is a common practice to avoid division by zero errors.
- The function is straightforward and easy to read.
- The function does not use type hints or any type annotations, which could be considered missing for type-aware coding practices.
(turn 1 tokens - input: 217, output: 105, total: 322)

> [assistant]: - The function 'get_first' is named correctly.
- The function has a docstring that explains its purpose.
- The function takes a list as an argument and returns the first element of the list, which is its intended behavior.
- The function is concise and to the point, which is a good sign.
- The function does not use type hints or any type annotations, which could be considered missing for type-aware coding practices.
(turn 2 tokens - input: 346, output: 88, total: 434)

> [assistant]: - "RelationalDatabase"
- "DatabaseDesign"
- "SQLQueries"
(turn 3 tokens - input: 455, output: 17, total: 472)

> === /stats ===
turn count: 3
cumulative input tokens: 1018
cumulative output tokens: 210
cumulative total tokens: 1228
serialized conversation-history length: 2335 chars
=== end /stats ===

> [assistant]: - The function 'add' is named correctly.
- The function has a docstring that explains its purpose.
- The function takes two arguments, 'a' and 'b', which suggests it could handle addition. It does not include a check for 'a' and 'b' being zero or negative, which is a common practice to avoid potential errors.
- The function is straightforward and easy to read.
- The function does not use type hints or any type annotations, which could be considered missing for type-aware coding practices.
(turn 4 tokens - input: 496, output: 105, total: 601)

> [assistant]: - No issues found.
(turn 5 tokens - input: 618, output: 6, total: 624)

> === /stats ===
turn count: 5
cumulative input tokens: 2132
cumulative output tokens: 321
cumulative total tokens: 2453
serialized conversation-history length: 3082 chars
=== end /stats ===
> === session ended ===
cumulative input tokens: 2132
cumulative output tokens: 321
turn count: 5
(.venv) PS C:\Users\User\Desktop\Data_260_HW-1\Part-2>
```

Per-turn token counts

Turn	Message	Input tokens	Output tokens	Total tokens
1	review divide(a, b)	217	105	322
2	review get_first(lst)	346	88	434
3	tags for a relational databases course	455	17	472
4	review add(a, b)	496	105	601
5	closing remark	618	6	624

	After turn 3	After turn 5
Turn count	3	5
Cumulative input tokens	1018	2132
Cumulative output tokens	210	321
Cumulative total tokens	1228	2453
Serialized history length (chars)	2335	3082

Did the model actually follow AGENT.md

Only partially. Across two separate runs, bullet points were used reliably for every code review response, but the model's handling of the finer rules was inconsistent. In one run, a code review response ended with a "No issues found." bullet appended after real issues had already been listed, directly contradicting the instruction. In another run, a plain closing remark that was not a code review request was still answered with a single bullet instead of the plain prose the instruction calls for on non review turns. The model follows the general shape of the instruction, using bullets for reviews, fairly reliably, but does not reliably distinguish when the rule should and should not apply, which is worth stating plainly rather than claiming full compliance.

Why is prior conversation context resent with every turn

The model is stateless between calls. It has no memory of earlier turns unless that content is physically included in the current request, so the client has to resend the full history, the system prompt plus every prior user and assistant message, on every single turn for the model to have any awareness of what was discussed before.

How is a system prompt different from a user message

A system prompt sets a persistent, session wide behavioral rule and is treated as configuration rather than something said within the conversation. A user message is an actual conversational turn the model is expected to respond to. Here, AGENT.md's bullet only rule is the system prompt governing every response, while each line typed by the user is a distinct turn.

Why do input tokens grow over a conversation

Because every turn's input is the previous turn's full input plus whatever new message was just added. This was observed directly in the table above: input tokens climbed from 217 to 618 across the 5 turns, tracking the growing history almost exactly.

What eventually limits that growth

The model's context window, the maximum number of tokens it can accept in a single call. Once the history plus a new message would exceed that limit, something has to give: older turns get truncated or summarized out of the history, or the call fails outright. In practice, cost and latency also motivate managing context well before that hard ceiling is ever reached.